

Business Domain & Problem Statement

Clinical Background

Heart disease remains the **leading cause of death globally**, claiming over 17 million lives each year (WHO, 2023). Early prediction through machine learning can enable timely clinical intervention, reduce hospitalisation costs, and most importantly — *save lives*. Risk factors span behavioural, metabolic, and socio-economic dimensions, making this a rich multi-dimensional classification problem.

Dataset — BRFSS 2015

The **Behavioral Risk Factor Surveillance System (BRFSS) 2015** dataset, sourced from the CDC and hosted on [Kaggle](#), contains responses from **253,680 Americans** collected via telephone surveys. A stratified sample of **13,000 rows** is used here for computational efficiency.

Attribute	Detail
Target Variable	heart_disease — Binary (0 = No, 1 = Yes)
Features	21 health indicators (binary, ordinal, continuous)
Class Imbalance	~91% Negative / ~9% Positive → SMOTE oversampling applied
Sample Size	10,000 rows (stratified random sample)

Project Objectives

1. Perform thorough **Exploratory Data Analysis** on health indicators.
2. Apply **Feature Selection** (Spearman + Mutual Information) to remove noise.
3. Evaluate the effect of **PCA dimensionality reduction** on model performance.
4. Train and tune a **Support Vector Machine (SVM)** via GridSearchCV.
5. Maximise **Recall for the positive class** (minimise missed patients).

Project Workflow

- ☒ 1. Problem Definition
- ☒ 2. Data Loading
- ☒ 3. Train-Test Split
- ☒ 4. EDA
- ☒ 5. Data Cleaning
- ☒ 6. Build Pipeline

```
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=0.95)),
    ("model", SVC())
])
```

- ☒ 7. Train Model
- ☒ 8. Evaluate
- ☒ 9. Save Everything

1. Library Imports

All required packages for ML pipeline

We import data handling (`pandas`, `numpy`), visualisation (`matplotlib`, `seaborn`), preprocessing, dimensionality reduction, SVM modelling, and evaluation tools from `scikit-learn` and `imbalanced-learn`.

```
import kagglehub
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
```

```

# preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

# Models
from sklearn.svm import LinearSVC, SVC
from sklearn.metrics import accuracy_score

# Dimensionality Reduction
from sklearn.feature_selection import mutual_info_classif
from sklearn.decomposition import PCA
from sklearn.model_selection import GridSearchCV

# Evaluation
from sklearn.metrics import (
    classification_report,
    accuracy_score,
    confusion_matrix,
    make_scorer,
    recall_score
)

# Deployment
import joblib
import os
from google.colab import userdata

RANDOM_STATE = 42
SAMPLE_SIZE = 13000
np.random.seed(RANDOM_STATE)

```

2. Download & Load Dataset

Kaggle BRFSS 2015 — Heart Disease Health Indicators

The dataset is downloaded via [kagglehub](#) and a stratified sample of **10,000 rows** is taken with a fixed random seed to ensure reproducibility across all experiments.

```

path = kagglehub.dataset_download("alexteboul/heart-disease-health-indicators-dataset")
print("Dataset downloaded to:", path)

df = pd.read_csv(f'{path}/heart_disease_health_indicators_BRFSS2015.csv')
df = df.sample(n=SAMPLE_SIZE, random_state=42) # Sample 10k rows for faster processing

print(f"Loaded file : {path}")
print(f"Shape      : {df.shape[0]:,} rows x {df.shape[1]} columns")

```

```

Using Colab cache for faster access to the 'heart-disease-health-indicators-dataset' dataset.
Dataset downloaded to: /kaggle/input/heart-disease-health-indicators-dataset
Loaded file : /kaggle/input/heart-disease-health-indicators-dataset
Shape      : 13,000 rows x 22 columns

```

Column Renaming— Replace cryptic abbreviations with human-readable names

```

def rename_health_columns(df):
    return df.rename(columns={
        "HeartDiseaseorAttack": "heart_disease",
        "HighBP": "high_blood_pressure",
        "HighChol": "high_cholesterol",
        "CholCheck": "cholesterol_check",
        "BMI": "bmi",
        "Smoker": "is_smoker",
        "Stroke": "had_stroke",
        "Diabetes": "diabetes_status",
        "PhysActivity": "physical_activity",
        "Fruits": "eats_fruits",
        "Veggies": "eats_vegetables",
        "HvyAlcoholConsump": "heavy_alcohol_consumption",
        "AnyHealthcare": "has_healthcare",
        "NoDocbcCost": "no_doctor_due_to_cost",
        "GenHlth": "general_health",
        "MentHlth": "mental_health_days",
    })

```

```

        "PhysHlth": "physical_health_days",
        "DiffWalk": "difficulty_walking",
        "Sex": "gender",
        "Age": "age_group",
        "Education": "education_level",
        "Income": "income_level",
    })

df = rename_health_columns(df)

```

2.2 Stratified Train/Test Split

To prevent data leakage, we split the data into training and testing sets before performing any exploratory analysis or feature engineering.

⚠ Best Practice: We split the data *before* EDA so that all statistics (correlations, distributions) are computed solely on the training set. This prevents inadvertently learning from test data. Stratification preserves class proportions.

```

# Define target and features
X = df.drop(columns=['heart_disease'])
y = df['heart_disease']

# Perform Stratified 80/20 Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y
)

# Apply dropna to the split data directly
train_mask = X_train.notna().all(axis=1) & y_train.notna()
X_train = X_train[train_mask]
y_train = y_train[train_mask]

# Note: drop_duplicates is intentionally NOT applied.
# BRFSS survey data contains many structurally identical rows because respondents
# share the same binary/ordinal health profile. These are NOT true data entry errors,
# so removing them would discard valid minority-class samples and harm recall.

print(f'Training set : {X_train.shape[0]:,} rows')
print(f'Test set      : {X_test.shape[0]:,} rows')

Training set : 10,400 rows
Test set      : 2,600 rows

```

3. Exploratory Data Analysis

Data Quality Check— Missing values, duplicates, and data integrity

3.1. We inspect for **null values** and **duplicate rows**. The BRFSS dataset is known to contain duplicates from repeated survey entries, which must be removed to prevent data leakage.

```

# Check for missing values
print(df.isnull().sum())

# Check for duplicate rows (common in this dataset)
print(f"Duplicates: {df.duplicated().sum()}")
print('')
drop_duplicates is intentionally NOT applied
BRFSS survey data contains many structurally identical rows because respondents
share the same binary/ordinal health profile. These are NOT true data entry errors,
so removing them would discard valid minority-class samples and harm recall.
'''

```

heart_disease	0
high_blood_pressure	0
high_cholesterol	0
cholesterol_check	0
bmi	0
is_smoker	0
had_stroke	0
diabetes_status	0

```

physical_activity      0
eats_fruits            0
eats_vegetables        0
heavy_alcohol_consumption 0
has_healthcare          0
no_doctor_due_to_cost  0
general_health          0
mental_health_days      0
physical_health_days    0
difficulty_walking      0
gender                 0
age_group               0
education_level         0
income_level            0
dtype: int64
Duplicates: 210

```

drop_duplicates is intentionally NOT applied
 BRFSS survey data contains many structurally identical rows because respondents share the same binary/ordinal health profile. These are NOT true data entry errors, so removing them would discard valid minority-class samples and harm recall.

3.2. Data Preview— First look at the cleaned, renamed DataFrame

```
df.head()
```

	heart_disease	high_blood_pressure	high_cholesterol	cholesterol_check	bmi	is_smoker	had_stroke	diabetes_status
219620	0.0	0.0	0.0	1.0	21.0	0.0	0.0	0.0
132821	0.0	1.0	1.0	1.0	28.0	0.0	0.0	0.0
151862	0.0	0.0	0.0	1.0	24.0	0.0	0.0	0.0
139717	0.0	0.0	0.0	1.0	27.0	1.0	0.0	0.0
239235	0.0	0.0	1.0	1.0	31.0	1.0	0.0	0.0

5 rows × 22 columns

3.3. Unique Value Inspection— Verify expected value ranges per feature

All features are expected to be categorical or ordinal integers. This check confirms there are no unexpected continuous values or encoding errors in binary columns.

```

for col in df.columns:
    print(f"column '{col}' -> has {df[col].nunique()} unique values")

```

```

column 'heart_disease' -> has 2 unique values
column 'high_blood_pressure' -> has 2 unique values
column 'high_cholesterol' -> has 2 unique values
column 'cholesterol_check' -> has 2 unique values
column 'bmi' -> has 65 unique values
column 'is_smoker' -> has 2 unique values
column 'had_stroke' -> has 2 unique values
column 'diabetes_status' -> has 3 unique values
column 'physical_activity' -> has 2 unique values
column 'eats_fruits' -> has 2 unique values
column 'eats_vegetables' -> has 2 unique values
column 'heavy_alcohol_consumption' -> has 2 unique values
column 'has_healthcare' -> has 2 unique values
column 'no_doctor_due_to_cost' -> has 2 unique values
column 'general_health' -> has 5 unique values
column 'mental_health_days' -> has 30 unique values
column 'physical_health_days' -> has 31 unique values
column 'difficulty_walking' -> has 2 unique values
column 'gender' -> has 2 unique values
column 'age_group' -> has 13 unique values
column 'education_level' -> has 6 unique values
column 'income_level' -> has 8 unique values

```

3.4. Feature Column Groups— Categorise features for the preprocessing pipeline

```

def categorize_features(df):

    target = 'heart_disease'

    binary_cols = []
    scale_cols = []

    for col in df.columns:
        if col == target:

```

```

if col == target:
    continue

if df[col].nunique() == 2 and set(df[col].unique()).issubset({0, 1}):
    binary_cols.append(col)
else:
    scale_cols.append(col)

return binary_cols, scale_cols

binary_cols = categorize_features(df)[0]
scale_cols = categorize_features(df)[1]

print(f"Binary Columns: {binary_cols}")
print(f"Scale Columns: {scale_cols}")

```

Binary Columns: ['high_blood_pressure', 'high_cholesterol', 'cholesterol_check', 'is_smoker', 'had_stroke', 'physical_activit
Scale Columns: ['bmi', 'diabetes_status', 'general_health', 'mental_health_days', 'physical_health_days', 'age_group', 'educa

```

# distribution of the classes in training dataset
targets = y_train.value_counts()
colors = ['lavender', 'skyblue']

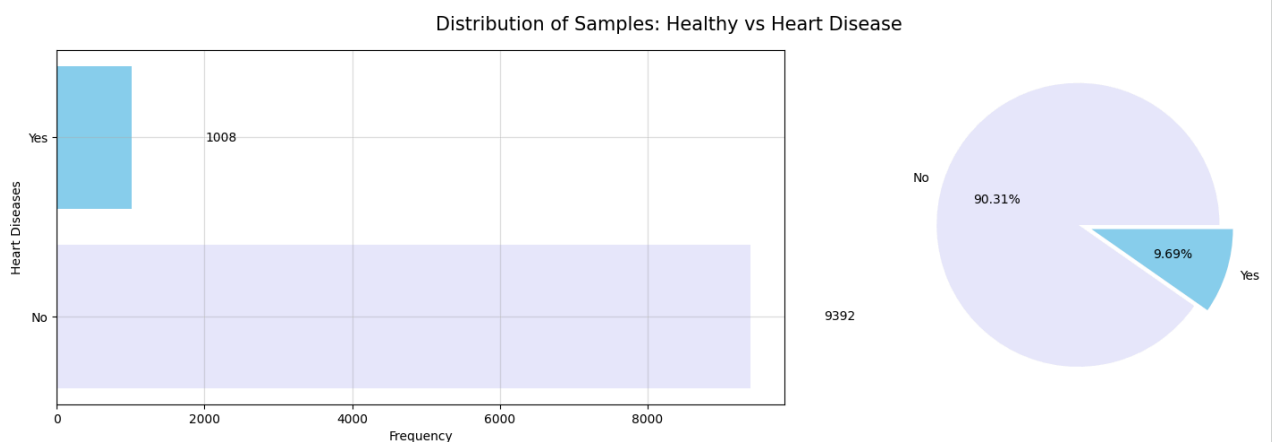
# visualization
figure, axes = plt.subplots(1,2, figsize=(15,5), gridspec_kw={'width_ratios':[1.5,1]})
axes[0].barh(y=targets.index, width=targets.values, color=colors)
axes[0].set_xlabel('Frequency')
axes[0].set_ylabel('Heart Diseases')
axes[0].set_yticks([0,1], ['No', 'Yes'])
axes[0].grid(alpha=0.4)

for index, values in enumerate(targets):
    axes[0].text(values+1000, index, str(values), va='center')

axes[1].pie(targets.values, labels=['No', 'Yes'], autopct='%0.2f%%', explode=(0.05*len(targets.index)), colors=colors)

figure.suptitle('Distribution of Samples: Healthy vs Heart Disease', fontsize=15)
plt.tight_layout(pad=1)
plt.show()

```



3.5. Feature Distributions— Histogram grid for all 21 features on the training set

```

# Distribution of Features
fig, axes = plt.subplots(4, 6, figsize=(22, 14))
axes = axes.flatten()

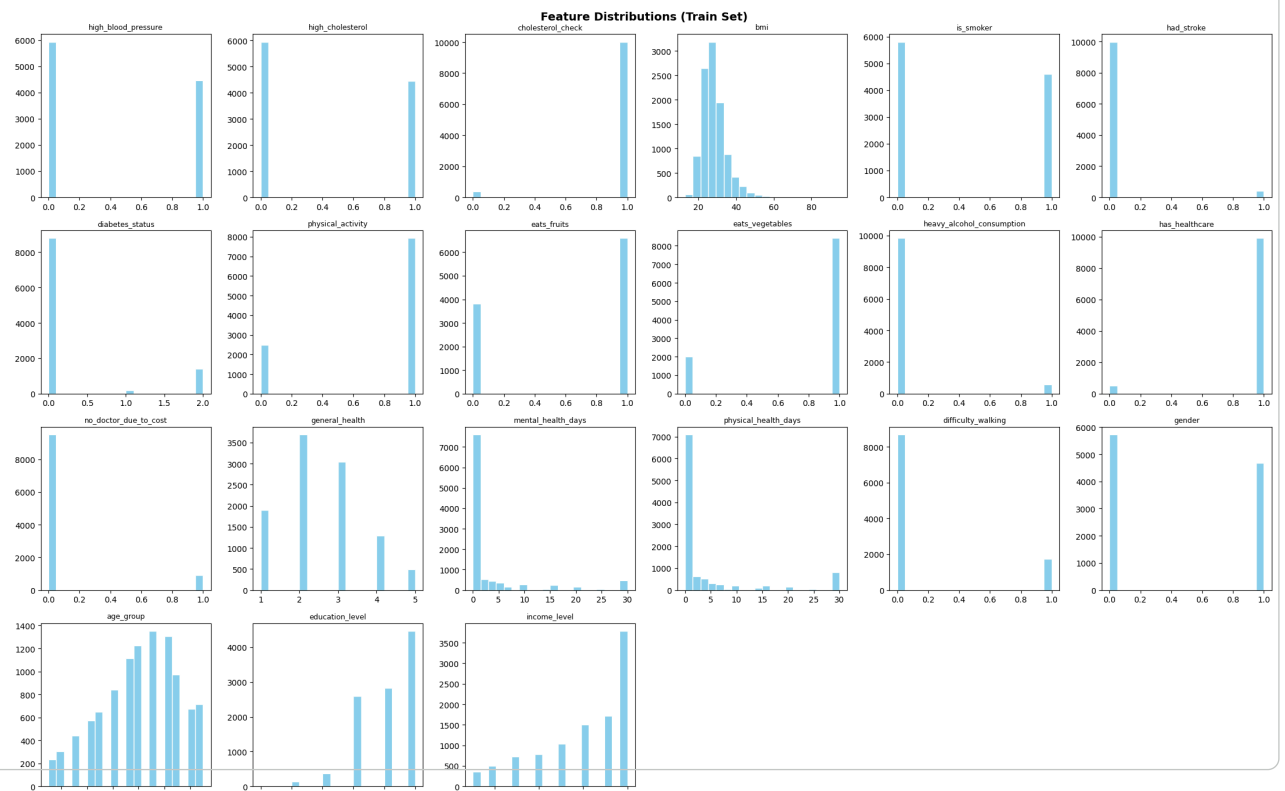
for i, col in enumerate(X_train.columns):
    axes[i].hist(X_train[col], bins=20, color='skyblue', edgecolor='white')
    axes[i].set_title(col, fontsize=9)

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Feature Distributions (Train Set)', fontsize=14, fontweight='bold')

```

```
plt.tight_layout()
plt.show()
```

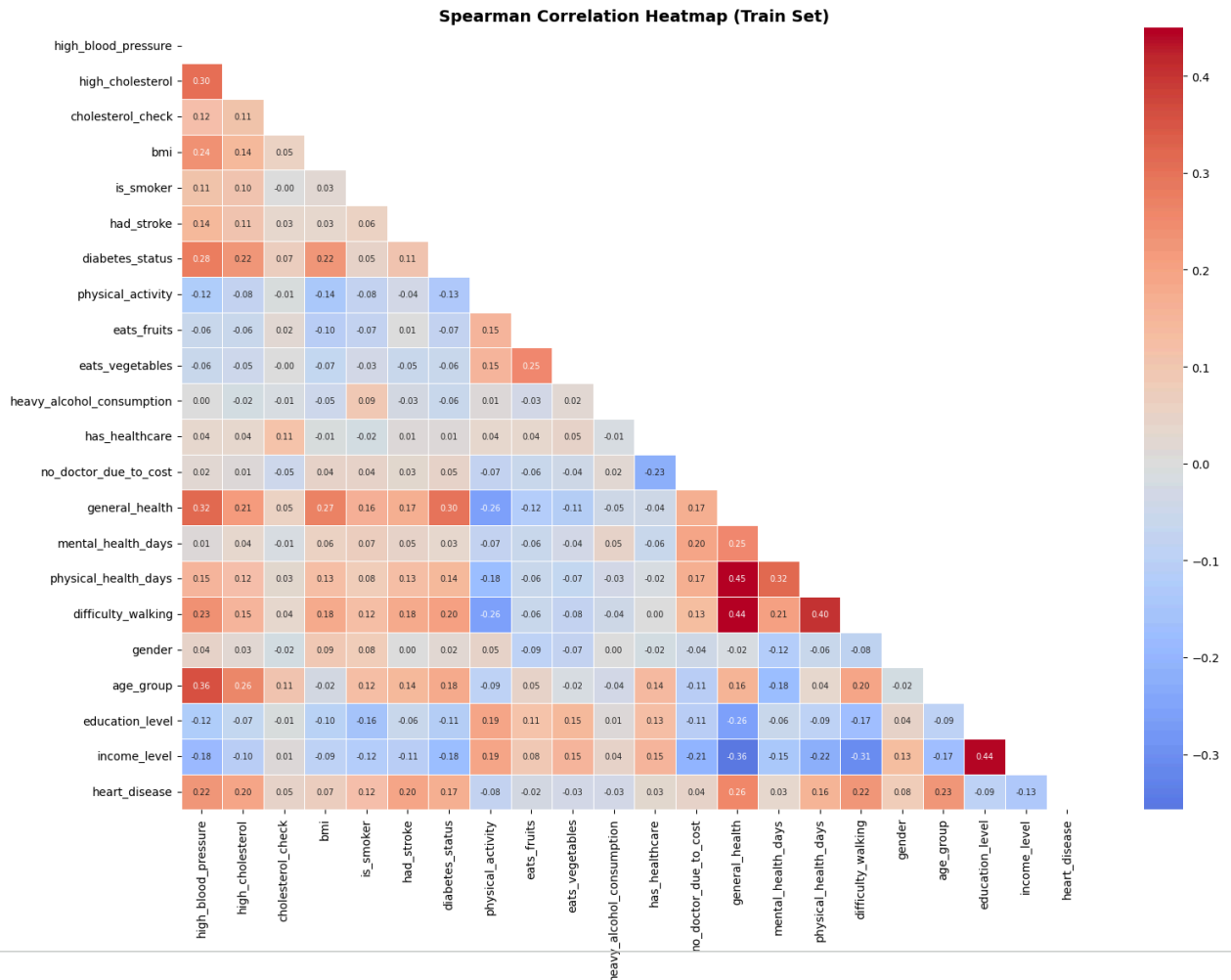


3.6. Spearman Correlation Heatmap— Rank-based correlation capturing non-linear monotonic relationships

We use **Spearman correlation** (instead of Pearson) because many features are ordinal or binary, for which rank-based measures are more appropriate.

```
# Correlation Heatmap
train_df = X_train.copy()
train_df['heart_disease'] = y_train.values

plt.figure(figsize=(16, 12))
corr_matrix = train_df.corr(method='spearman')
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f',
            cmap='coolwarm', center=0, linewidths=0.5, annot_kws={"size": 7})
plt.title('Spearman Correlation Heatmap (Train Set)', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



3.7. Feature vs Target Analysis— Boxplots comparing key feature distributions across classes

```
import matplotlib.pyplot as plt
import seaborn as sns

plot_df = X_train.copy()
plot_df['heart_disease'] = y_train.values

features_to_plot = [
    'general_health',
    'age_group',
    'physical_health_days',
    'bmi',
    'income_level'
]

n_features = len(features_to_plot)
n_cols = 3
n_rows = (n_features + n_cols - 1) // n_cols

fig, axes = plt.subplots(n_rows, n_cols, figsize=(5 * n_cols, 4 * n_rows))
axes = axes.flatten()

for i, feature in enumerate(features_to_plot):
    sns.boxplot(x='heart_disease', y=feature, data=plot_df, ax=axes[i], palette='pastel')
    axes[i].set_title(f'{feature.replace("_", " ").title()} vs. Heart Disease', fontsize=12)
    axes[i].set_xlabel('Heart Disease (0=No, 1=Yes)')
    axes[i].set_ylabel(feature.replace("_", " ").title())
    axes[i].tick_params(axis='x', rotation=0)

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
plt.suptitle('Feature Distributions by Heart Disease Status', y=1.02, fontsize=16, fontweight='bold')
plt.show()

# Insights from these boxplots:
# - General Health: Individuals with heart disease tend to report poorer general health (higher values).
# - Age Group: Older age groups show a higher prevalence of heart disease.
# - Physical Health Days: Patients with heart disease report significantly more days of poor physical health.
# - BMI: While there's an overlap, the median BMI for individuals with heart disease appears slightly higher, indicating a p
```

- Income Level: Lower income levels are associated with a higher incidence of heart disease, suggesting socio-economic fac

```
/tmp/ipykernel_18963/2970997780.py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.boxplot(x='heart_disease', y=feature, data=plot_df, ax=axes[i], palette='pastel')
```

```
/tmp/ipykernel_18963/2970997780.py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.boxplot(x='heart_disease', y=feature, data=plot_df, ax=axes[i], palette='pastel')
```

```
/tmp/ipykernel_18963/2970997780.py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.boxplot(x='heart_disease', y=feature, data=plot_df, ax=axes[i], palette='pastel')
```

```
/tmp/ipykernel_18963/2970997780.py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

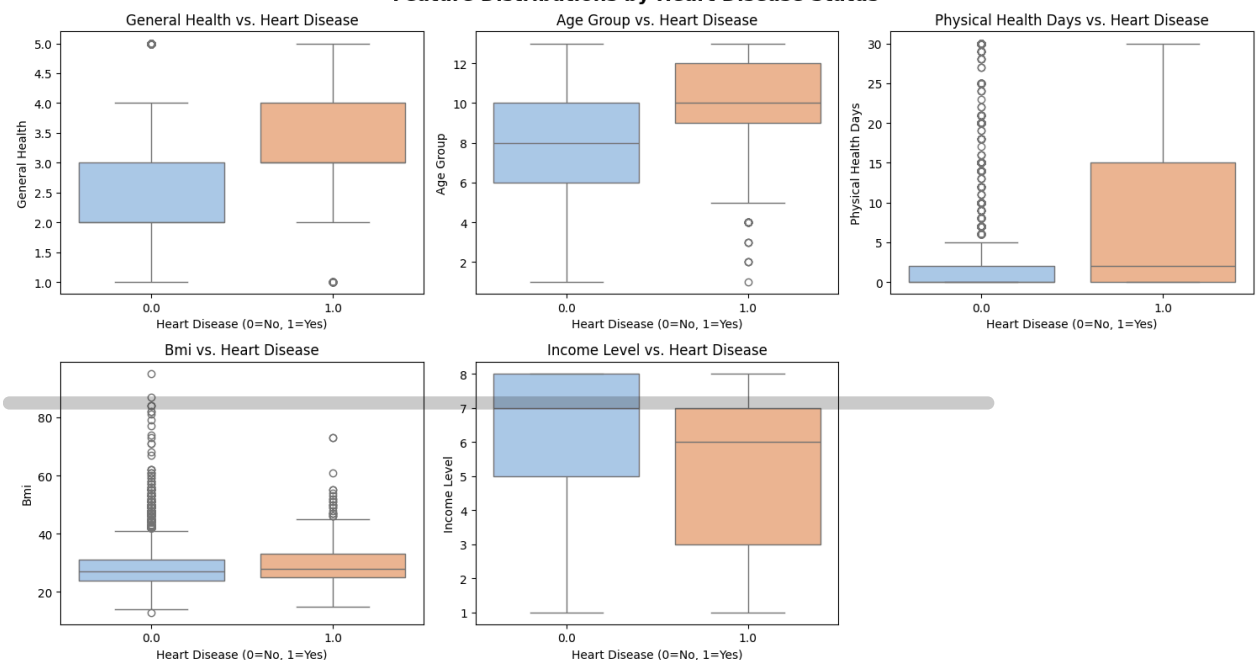
```
sns.boxplot(x='heart_disease', y=feature, data=plot_df, ax=axes[i], palette='pastel')
```

```
/tmp/ipykernel_18963/2970997780.py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.boxplot(x='heart_disease', y=feature, data=plot_df, ax=axes[i], palette='pastel')
```

Feature Distributions by Heart Disease Status



3.8. Class Balance Analysis— Quantify the imbalance and identify top predictive features

The dataset is highly **imbalanced** (~91% negative class). We will apply **SMOTE** within the training pipeline to synthetically oversample the minority class and improve recall for heart disease detection.

```
# Class Balance + Outliers
print("Class Distribution in Train Set:")
print(y_train.value_counts(normalize=True).mul(100).round(2).astype(str) + '%')

print("\nTop correlated features with heart_disease (Spearman):")
corr_target = train_df.corr(method='spearman')['heart_disease'].drop('heart_disease')
print(corr_target.abs().sort_values(ascending=False).head(8))
```

```
Class Distribution in Train Set:
heart_disease
0.0    90.31%
1.0     9.69%
Name: proportion, dtype: object
```

```
Top correlated features with heart_disease (Spearman):
general_health    0.257926
age_group         0.225513
high_blood_pressure 0.224785
difficulty_walking 0.222642
high_cholesterol   0.197172
had_stroke         0.196299
diabetes_status    0.174110
```



```
physical_health_days    0.156365
Name: heart_disease, dtype: float64
```

4. Data Preprocessing & Cleaning

Scaling, encoding, class balancing, and baseline model evaluation

Baseline Pipeline — All Features (No Dim Reduction)— Benchmark result with full 21-feature set

The **baseline pipeline** applies:

- **MinMaxScaler** on ordinal/continuous features
- **Passthrough** on binary features (already 0/1 encoded)
- **SMOTE** to handle class imbalance on training data only
- **LinearSVC** — fast, effective for high-dimensional linearly separable problems

This serves as the *comparison baseline* before dimensionality reduction is applied.

```
preprocessor = ColumnTransformer([
    ('scale', MinMaxScaler(), scale_cols),
    ('pass', 'passthrough', binary_cols)
])

from imblearn.pipeline import Pipeline as ImbPipeline

pipeline = ImbPipeline([
    ("preprocessor", preprocessor),
    ("smote", SMOTE(random_state=42)),
    ("classifier", LinearSVC(class_weight='balanced',
                             random_state=42,
                             max_iter=5000,
                             dual=False ))
])
```

```
pipeline.fit(X_train, y_train)
pred1 = pipeline.predict(X_test)
```

```
class_report = classification_report(y_test, pred1)
conf_matrix = confusion_matrix(y_test, pred1)
```

```
print("Accuracy: {:.2f}%".format(accuracy_score(y_test, pred1) * 100))
print("Classification Report:")
print(class_report)
```

```
Accuracy: 76.65%
Classification Report:
              precision    recall  f1-score   support

     0.0         0.98      0.76      0.85        2348
     1.0         0.27      0.85      0.41         252

 accuracy          0.77        2600
 macro avg         0.63        0.80        0.63        2600
 weighted avg         0.91        0.77        0.81        2600
```

5. Dimensionality Reduction

Feature selection via Spearman & Mutual Information, followed by PCA

Feature Importance: Spearman & Mutual Information— Dual-criterion feature ranking

Two complementary metrics assess feature relevance:

- **Spearman Correlation** — captures monotonic relationships between each feature and the target
- **Mutual Information** — captures any statistical dependency, including non-linear

5.1. Features scoring low on *both* metrics are candidates for removal.

```
# --- Spearman ---
spearman_corr = X_train.copy()
spearman_corr['heart_disease'] = y_train
```

```
spearman = spearman_corr.corr(method='spearman')['heart_disease'].drop('heart_disease')

# --- Mutual Information ---
mi = mutual_info_classif(X_train, y_train, random_state=42)

mi_series = pd.Series(mi, index=X_train.columns)

# --- Combine ---
comparison_df = pd.DataFrame({
    'Spearman': spearman,
    'Mutual_Info': mi_series
})

comparison_df = comparison_df.sort_values(by='Mutual_Info', ascending=False)

print(comparison_df)
```

	Spearman	Mutual_Info
general_health	0.257926	0.041985
age_group	0.225513	0.030562
high_blood_pressure	0.224785	0.027525
difficulty_walking	0.222642	0.021271
high_cholesterol	0.197172	0.019448
physical_health_days	0.156365	0.017436
is_smoker	0.117955	0.014164
diabetes_status	0.174110	0.013554
had_stroke	0.196299	0.012335
income_level	-0.134852	0.009406
gender	0.083030	0.007308
education_level	-0.087015	0.006449
physical_activity	-0.075814	0.005740
bmi	0.067624	0.005140
cholesterol_check	0.048846	0.002472
eats_fruits	-0.024164	0.002304
no_doctor_due_to_cost	0.040227	0.000630
has_healthcare	0.029804	0.000482
heavy_alcohol_consumption	-0.028336	0.000000
eats_vegetables	-0.032238	0.000000
mental_health_days	0.032423	0.000000

5.2. Selection Thresholds & Features to Drop

Spearman Threshold

Drop if $|p| < 0.06$

Mutual Information Threshold

Drop if $MI \leq 0.005$

Features dropped: `heavy_alcohol_consumption`, `eats_fruits`, `eats_vegetables`, `cholesterol_check`, `mental_health_days`, `has_healthcare`

```
def get_columns_to_drop(comparison_df, spearman_thresh=0.06, mi_thresh=0.005):
    cols = comparison_df[
        (comparison_df['Spearman'].abs() < spearman_thresh) &
        (comparison_df['Mutual_Info'] <= mi_thresh)
    ].index.tolist()

    print(f"Automatically identified {len(cols)} columns to drop: {cols}")
    return cols

columns_to_drop = get_columns_to_drop(comparison_df)
```

Automatically identified 7 columns to drop: ['cholesterol_check', 'eats_fruits', 'no_doctor_due_to_cost', 'has_healthcare', 'heavy_alcohol_consumption', 'eats_vegetables', 'mental_health_days']

```
X_train.drop(columns=columns_to_drop, inplace=True, errors='ignore')
X_test.drop(columns=columns_to_drop, inplace=True, errors='ignore')
```

5.3. Updated Feature Column Groups— After dropping low-importance features

3. Automatically re-categorize remaining columns for the pipeline

```
binary_cols = categorize_features(X_train)[0]
scale_cols = categorize_features(X_train)[1]

print(f"Remaining Binary Columns: {binary_cols}")
print(f"Remaining Scale Columns: {scale_cols}")
```

Remaining Binary Columns: ['high_blood_pressure', 'high_cholesterol', 'is_smoker', 'had_stroke', 'physical_activity', 'diffic
Remaining Scale Columns: ['bmi', 'diabetes_status', 'general_health', 'physical_health_days', 'age_group', 'education_level',

6. Vector Machine (SVM) & hyperparameter tuning

Experiment A — SVM Without PCA— After feature selection, before dimensionality reduction

Using the **13 selected features** (after removing low-importance ones), we train a `LinearSVC` with `SMOTE` balancing. This establishes the effect of feature selection alone.

```
# Update column lists to match the columns remaining after feature selection
current_scale_cols = [col for col in scale_cols if col in X_train.columns]
current_binary_cols = [col for col in binary_cols if col in X_train.columns]

preprocessor = ColumnTransformer([
    ('scale', MinMaxScaler(), current_scale_cols),
    ('pass', 'passthrough', current_binary_cols)
])

pipeline = ImbPipeline([
    ("preprocessor", preprocessor),
    ("smote", SMOTE(random_state=42)),
    ("classifier", LinearSVC(class_weight='balanced',
                             random_state=42,
                             max_iter=5000,
                             dual=False ))
])

# Train and evaluate
pipeline.fit(X_train, y_train)
pred1 = pipeline.predict(X_test)
```

```
class_report = classification_report(y_test, pred1)
conf_matrix = confusion_matrix(y_test, pred1)
```

```
print("Accuracy: {:.2f}%".format(accuracy_score(y_test, pred1) * 100))
print("Classification Report:")
print(class_report)
```

```
Accuracy: 76.42%
Classification Report:
```

	precision	recall	f1-score	support
0.0	0.98	0.75	0.85	2348
1.0	0.27	0.86	0.41	252
accuracy			0.76	2600
macro avg	0.63	0.81	0.63	2600
weighted avg	0.91	0.76	0.81	2600

Experiment B — SVM With PCA— Adding PCA (95% variance retained) to the pipeline

PCA is inserted between preprocessing and SMOTE. With **95% variance retention**, PCA compresses correlated features into orthogonal principal components, potentially reducing overfitting and improving generalisation.

```
preprocessor = ColumnTransformer([
    ('scale', MinMaxScaler(), scale_cols),
    ('pass', 'passthrough', binary_cols)
])

pipeline = ImbPipeline([
    ("preprocessor", preprocessor),
    ('pca', PCA(n_components=0.95, random_state=42)),
    ("smote", SMOTE(random_state=42)),
    ("classifier", LinearSVC(class_weight='balanced',
                             random_state=42,
                             max_iter=5000,
                             dual=False ))
])

pipeline.fit(X_train, y_train)
pred1 = pipeline.predict(X_test)
```

```
class_report = classification_report(y_test, pred1)
conf_matrix = confusion_matrix(y_test, pred1)
```

```
print("Accuracy: {:.2f}%".format(accuracy_score(y_test, pred1) * 100))
print("Classification Report:")
print(class_report)
```

```
Accuracy: 75.38%
Classification Report:
              precision    recall  f1-score   support

     0.0         0.98      0.74      0.84       2348
     1.0         0.26      0.86      0.40        252

 accuracy          0.62      0.80      0.75       2600
  macro avg          0.62      0.80      0.62       2600
 weighted avg          0.91      0.75      0.80       2600
```

PCA Deep-Dive— Inspect components, explained variance, and transformed feature space

```
pca = pipeline.named_steps['pca']
print("Number of components:", pca.n_components_)
```

```
Number of components: 11
```

```
pc_names = [f"PC{i+1}" for i in range(pca.n_components_)]
print(pc_names)
```

```
['PC1', 'PC2', 'PC3', 'PC4', 'PC5', 'PC6', 'PC7', 'PC8', 'PC9', 'PC10', 'PC11']
```

```
X_pca = pipeline.named_steps['pca'].transform(
    pipeline.named_steps['preprocessor'].transform(X_train)
)

df_pca = pd.DataFrame(
    X_pca,
    columns=[f"PC{i+1}" for i in range(X_pca.shape[1])]
)

print(df_pca.shape)
df_pca.head()
```

```
(10400, 11)
```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	PC10	PC11
0	-0.373457	0.512476	-0.639781	0.151587	-0.621558	0.123611	-0.116922	0.105261	0.006478	-0.032776	-0.001677
1	-0.876245	-0.450893	-0.102535	0.079227	0.081258	0.039160	0.025477	0.202922	0.053433	-0.020946	0.028465
2	-0.759422	0.379213	-0.197935	-0.384143	-0.047976	0.166208	-0.006043	-0.117570	-0.164323	0.177336	-0.036290
3	0.051799	-0.647552	-0.170574	-0.509566	0.369590	-0.807772	-0.176715	0.184271	0.104015	0.177801	-0.048328
4	1.190876	-0.314919	0.213181	0.186555	-0.113185	-0.616159	0.781502	-0.209158	0.204701	-0.312431	0.024024

```
print(pca.explained_variance_ratio_)
```

```
[0.24047203 0.1468784  0.12892281 0.10871688 0.09178166 0.07202725
 0.05412909 0.04112637 0.03337574 0.02737813 0.01997897]
```

The plot above shows the cumulative explained variance. We can see that with just a few principal components, a significant portion of the variance in the original data is captured. For instance, with 11 components, 95% of the variance is explained.

Hyperparameter Tuning — GridSearchCV— Systematic search over SVM kernels and regularisation parameters

GridSearchCV with **5-fold cross-validation** optimises over:

- **Kernel:** RBF (non-linear boundary) vs. Linear (simple boundary)
- **C:** Regularisation strength $\in \{0.1, 1, 5, 10\}$
- **Gamma:** RBF width $\in \{\text{scale}, 0.01, 0.05\}$

Scoring metric: Recall on class 1 (positive/patient) — prioritising *sensitivity* in a medical context to minimise missed diagnoses.

```
# Preprocessing pipeline
# Standard scaling applied to all features
preprocessor = ColumnTransformer([
```

```

        ('scale', StandardScaler(), scale_cols + binary_cols)
    ])

# Build ML pipeline
pipeline_svm = ImbPipeline([
    ("preprocessor", preprocessor),          # Feature scaling
    ("smote", SMOTE(random_state=42)),      # Handle class imbalance
    ("classifier", SVC(class_weight='balanced', random_state=42)) # SVM model
])

```

```

# Hyperparameter search space
param_grid = [
    # RBF Kernel (non-linear decision boundary)
    {
        'classifier__kernel': ['rbf'],
        'classifier__C': [0.1, 1, 5, 10],
        'classifier__gamma': ['scale', 0.01, 0.05]
    },

    # Polynomial Kernel (non-linear decision boundary)
    # لو مضحي باللاب اعملها
    {
        'classifier__kernel': ['poly'],
        'classifier__C': [0.1, 1, 5],
        'classifier__degree': [2, 3, 4],
        'classifier__gamma': ['scale', 0.01, 0.05]
    }
]

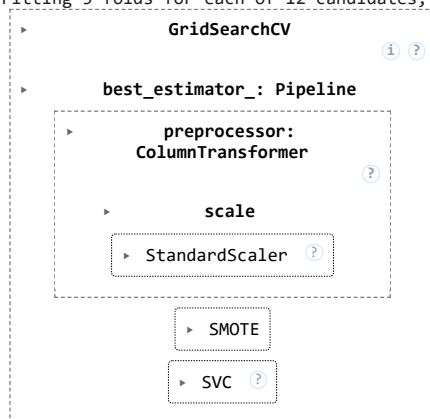
# Custom scoring metric (Medical Recall)
# Focus on detecting positive class (patients = class 1.0)
medical_recall = make_scorer(recall_score, pos_label=1.0)

# Grid Search Cross Validation
grid_search = GridSearchCV(
    pipeline_svm,
    param_grid=param_grid,
    cv=5,
    scoring=medical_recall,
    n_jobs=-1,
    verbose=1
)

# Train the model
grid_search.fit(X_train, y_train)

```

Fitting 5 folds for each of 12 candidates, totalling 60 fits



```

# Make predictions using best model
y_pred = grid_search.best_estimator_.predict(X_test)

```

7. Model Evaluations

Comparison Accuracy, recall, classification reports, and confusion matrices

GridSearch SVM — Full Evaluation— Accuracy · Recall · F1 · Precision

```
# Model Evaluation
print("\nGrid Search SVM – Evaluation")

# Accuracy
print(f"\nAccuracy: {accuracy_score(y_test, y_pred):.4f}")

# Recall for positive class (patients)
print(f"Recall (class 1 - patients): {recall_score(y_test, y_pred, pos_label=1.0):.4f}")

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
\nGrid Search SVM – Evaluation
```

```
Accuracy: 0.7504
```

```
Recall (class 1 - patients): 0.8889
```

```
Classification Report:
```

	precision	recall	f1-score	support
0.0	0.98	0.74	0.84	2348
1.0	0.27	0.89	0.41	252
accuracy			0.75	2600
macro avg	0.62	0.81	0.63	2600
weighted avg	0.91	0.75	0.80	2600

```
<>:2: SyntaxWarning: invalid escape sequence '\G'
<>:2: SyntaxWarning: invalid escape sequence '\G'
/tmp/ipykernel_18963/4093246376.py:2: SyntaxWarning: invalid escape sequence '\G'
print("\nGrid Search SVM – Evaluation")
```

✧ Confusion Matrix — GridSearch SVM

In a medical classification context, the four quadrants carry asymmetric costs:

- **True Negatives** — Correctly identified healthy individuals
- **False Positives** — Healthy flagged as ill (unnecessary follow-up)
- **False Negatives** — Sick patients missed (highest clinical risk)
- **True Positives** — Correctly identified patients

Minimising *False Negatives* is the primary objective — hence optimising for Recall.

```
# Confusion Matrix
print("Confusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)

# Detailed explanation
print(f"\nTrue Negatives (Correct healthy): {cm[0][0]}")
print(f"False Positives (Wrong patient): {cm[0][1]}")
print(f"False Negatives (Missed patient): {cm[1][0]}")
print(f"True Positives (Correct patient): {cm[1][1]}")
```

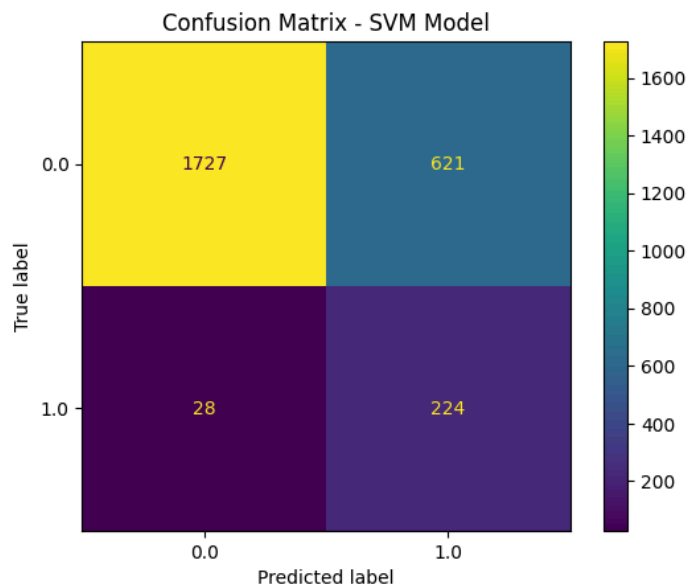
```
Confusion Matrix:
```

```
[[1727  621]
 [  28 224]]
```

```
True Negatives (Correct healthy): 1727
False Positives (Wrong patient):  621
False Negatives (Missed patient):  28
True Positives (Correct patient): 224
```

```
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)

plt.title("Confusion Matrix - SVM Model")
plt.show()
```



★ Best Hyperparameters Found— Optimal configuration from GridSearchCV

```
# Best Model Parameters
print("Best Hyperparameters")

for param, value in grid_search.best_params_.items():
    print(f"{param}: {value}")
```

```
Best Hyperparameters
classifier__C: 0.1
classifier__gamma: 0.01
classifier__kernel: rbf
```

8. Save the Model

Persist the best trained pipeline for deployment and future inference

Export Best Model & Artefacts— Using joblib for efficient serialisation

We persist three artefacts:

- **Best Pipeline** — Full GridSearchCV estimator (preprocessor + SMOTE + tuned SVM)
- **GridSearch Object** — Contains CV results and all evaluated parameter combinations
- **Feature Columns** — The exact 13 feature names used during training

These files allow *zero-code reloading* of the trained model for production inference.

```
feature_cols = X_train.columns.tolist()
print(feature_cols)
print("Model Have",len(feature_cols),"Features")
```

```
['high_blood_pressure', 'high_cholesterol', 'bmi', 'is_smoker', 'had_stroke', 'diabetes_status', 'physical_activity', 'general_health']
Model Have 14 Features
```

```
# — Create output directory —————
save_dir = "saved_models"
os.makedirs(save_dir, exist_ok=True)

# — Save best pipeline (preprocessor + SMOTE + best SVM) —————
best_pipeline = grid_search.best_estimator_
joblib.dump(best_pipeline, f"{save_dir}/best_svm_pipeline.pkl")
print(f"Best pipeline saved → {save_dir}/best_svm_pipeline.pkl")

# — save features columns
joblib.dump(feature_cols, f"{save_dir}/feature_cols.pkl")

# — Save full GridSearchCV object (CV results + all params) —————
joblib.dump(grid_search, f"{save_dir}/grid_search_object.pkl")
print(f"GridSearch object saved → {save_dir}/grid_search_object.pkl")

# — Summary —————
```

```
print("\n Saved artefacts:")
for f in os.listdir(save_dir):
    size = os.path.getsize(f"{save_dir}/{f}")
    print(f"    {f:40s}    {size:>8,} bytes")
```

Best pipeline saved → saved_models/best_svm_pipeline.pkl
 GridSearch object saved → saved_models/grid_search_object.pkl

```
Saved artefacts:
feature_cols.pkl                241 bytes
grid_search_object.pkl          1,562,095 bytes
best_svm_pipeline.pkl           1,557,913 bytes
```

Reload & Verify— Confirm the saved model reproduces identical predictions

```
import joblib
import numpy as np
from sklearn.metrics import accuracy_score

# — Reload model —————
loaded_pipeline = joblib.load("saved_models/best_svm_pipeline.pkl")
print("Model loaded successfully")
print(f"    Type : {type(loaded_pipeline)}")
print(f"    Steps : {[s[0] for s in loaded_pipeline.steps]}")

# — Verify predictions match —————
y_pred_loaded = loaded_pipeline.predict(X_test)
Recall = recall_score(y_test, y_pred_loaded)
print(f"Reloaded model Recall : {(Recall)*100:.2f} %")
print("Predictions are reproducible and model is ready for deployment.")
```

```
Model loaded successfully
    Type : <class 'imblearn.pipeline.Pipeline'>
    Steps : ['preprocessor', 'smote', 'classifier']
Reloaded model Recall : 88.89 %
Predictions are reproducible and model is ready for deployment.
```

✧ Automatically Model Uploading to GitHub

✧ Set up GitHub Personal Access Token (PAT)

To push to a private GitHub repository, you'll need a [Personal Access Token \(PAT\)](#).

```
GH_TOKEN = userdata.get('GH_TOKEN')
```

```
repo_owner = "AbdoAhmedAbdelmonem"
repo_name = "MetalHeart"
repo_url = f"https://{repo_owner}:{GH_TOKEN}@github.com/{repo_owner}/{repo_name}.git"

import os

if not os.path.exists(repo_name):
    !git clone $repo_url

%cd $repo_name

!git pull origin main --rebase

!mkdir -p "Saved Models"

!rm -rf "Saved Models"/*

!cp -r /content/saved_models/* "Saved Models/"

!git config user.name "AbdoAhmedAbdelmonem"
!git config user.email "tokyo9900777@gmail.com"

!git add .

!git diff --cached --quiet || git commit -m "Update Saved Models from Colab"

!git push origin main
```


Cloning into 'MetalHeart'...